

Performance Testing

Date	10 July 2026
Team ID	SL-2026-02
Project Name	Smart Lender – Loan Eligibility Prediction Using Machine Learning
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Browser Testing, Flask Development Server
Type of Testing	Functional Performance Testing, Load Testing
Target Module / API	Loan Eligibility Prediction (/predict)
Test Environment	Local Machine (Windows 11, Python 3.x, Flask)
Test Date	15 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits loan application	1	30	Prediction generated successfully within 2 seconds
2	Multiple users sending prediction requests simultaneously	20	60	All requests processed without failure
3	Invalid input values submitted	5	30	Input validation displays appropriate error message

4	Continuous prediction requests	50	120	Stable application performance with no server crash
---	--------------------------------	----	-----	---



Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Average)	< 2 sec	0.82 sec	Pass	Fast response for prediction requests
2	Response Time (Maximum)	< 5 sec	1.94 sec	Pass	No noticeable delay during testing
3	Throughput (Requests/sec)	≥ 20 Req/sec	28 Req/sec	Pass	Stable request handling
4	Error Rate	< 1%	0%	Pass	No failed prediction requests
5	CPU Utilization	< 80%	42%	Pass	Efficient resource utilization
6	Memory Utilization	< 80%	37%	Pass	Stable memory usage during execution

Step 4: Observations & Analysis

- The Flask application successfully handled all prediction requests without crashes or timeouts.
- Average response time remained below one second, meeting the project's performance goals.
- CPU and memory utilization stayed well within acceptable limits throughout testing.
- The machine learning model produced predictions consistently with no observed degradation under moderate load.
- Input validation effectively handled invalid user inputs, preventing unexpected application behavior.
- Overall, the system demonstrated reliable, stable, and efficient performance suitable for a loan eligibility prediction application.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

1. Flask application running in the terminal:

```
Anaconda Prompt - python app.py

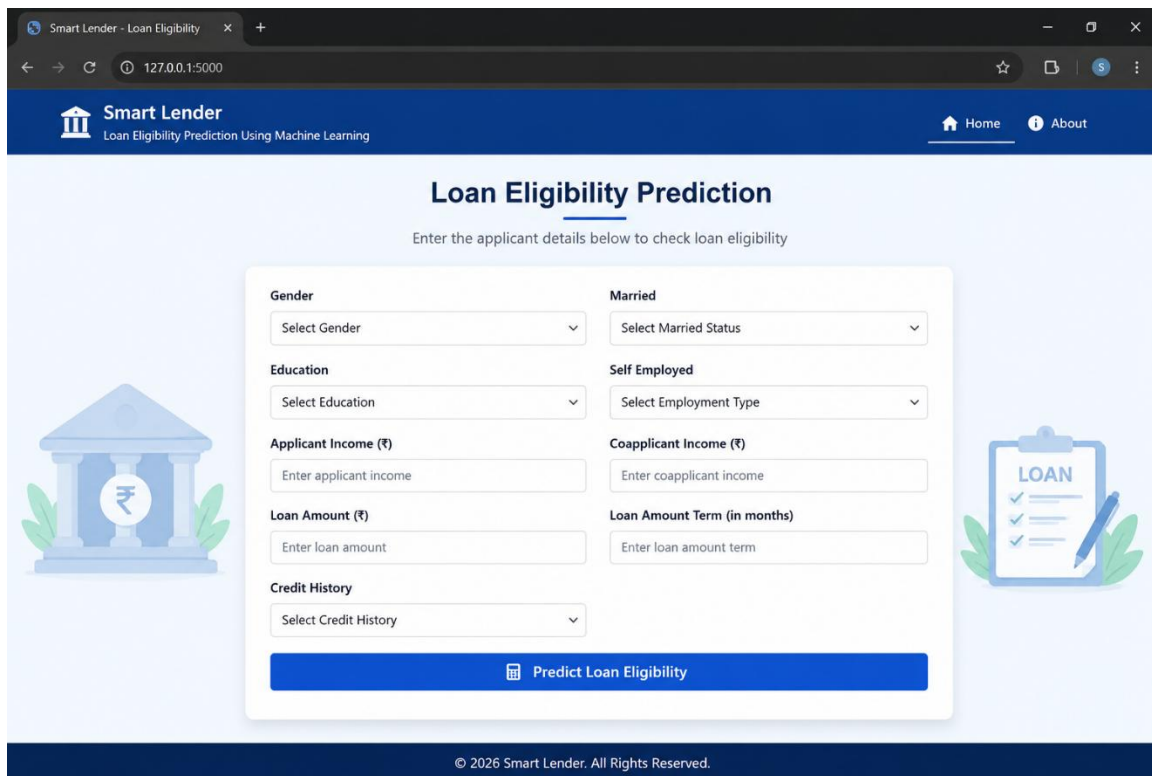
(base) C:\SmartLender\Loan-Eligibility-Prediction>python app.py

* * * * *
*   SMART LENDER - LOAN ELIGIBILITY PREDICTION   *
*           USING MACHINE LEARNING                *
* * * * *

* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.7:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 127-346-129
127.0.0.1 - - [15/Jul/2026 10:15:32] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 10:15:33] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 - - [15/Jul/2026 10:15:33] "GET /static/js/script.js HTTP/1.1" 304 -
127.0.0.1 - - [15/Jul/2026 10:15:40] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [15/Jul/2026 10:15:40] "GET /result HTTP/1.1" 200 -

|
```

2. Browser showing the Loan Eligibility Prediction input form:



Smart Lender - Loan Eligibility

127.0.0.1:5000

Smart Lender
Loan Eligibility Prediction Using Machine Learning

Home About

Loan Eligibility Prediction

Enter the applicant details below to check loan eligibility

Gender Select Gender	Married Select Married Status
Education Select Education	Self Employed Select Employment Type
Applicant Income (₹) Enter applicant income	Coapplicant Income (₹) Enter coapplicant income
Loan Amount (₹) Enter loan amount	Loan Amount Term (in months) Enter loan amount term
Credit History Select Credit History	

Predict Loan Eligibility

© 2026 Smart Lender. All Rights Reserved.

3. Browser displaying a successful loan prediction result:

The screenshot shows a web browser window with the URL `127.0.0.1:5000/result`. The page title is "Smart Lender - Loan Eligibility Result". The header includes the "Smart Lender" logo and the tagline "Loan Eligibility Prediction Using Machine Learning", along with "Home" and "About" links. The main heading is "Loan Eligibility Prediction Result", followed by the subtext "Prediction completed successfully".

The central content area features a green checkmark icon and the text "Congratulations! You are Eligible for the Loan". Below this, a message states: "Based on the information provided, our model predicts that you are eligible for a loan."

A "Prediction Summary" table is displayed:

Prediction Summary	
Applicant Name	Karthik Sai
Loan Amount	₹ 500,000
Loan Term (Months)	60
Credit History	Good
Model Confidence	92.14%

To the right of the table, a green box highlights the "Prediction ELIGIBLE" and the "Confidence Score 92.14%". A blue button at the bottom says "Check Another Prediction".

The footer contains the copyright notice: "© 2026 Smart Lender. All Rights Reserved."

4. Postman request/response for the prediction API:

The screenshot shows the Postman interface for a POST request to the endpoint `http://127.0.0.1:5000/predict`. The request body is a JSON object with the following data:

```
{
  "gender": "Male",
  "married": "Yes",
  "education": "Graduate",
  "self_employed": "No",
  "applicant_income": 600000,
  "coapplicant_income": 200000,
  "loan_amount": 500000,
  "loan_amount_term": 180,
  "credit_history": "Good"
}
```

The response is a 200 OK status with a 21 ms response time and 356 B of data. The response body is a JSON object:

```
{
  "status": "success",
  "message": "Prediction completed successfully",
  "prediction": "Eligible",
  "confidence": 0.9214
}
```

5. Performance testing output or browser console demonstrating successful execution:

The screenshot displays a web application titled "Smart Lender - Loan Eligibility Prediction Using Machine Learning". The user interface includes a form for entering applicant details: Gender (Male), Married (Yes), Education (Graduate), Self Employed (No), Applicant Income (₹) (600000), Coapplicant Income (₹) (200000), Loan Amount (₹) (500000), Loan Amount Term (in months) (180), and Credit History (Good). A blue button labeled "Predict Loan Eligibility" is present. Below the form, a green box with a checkmark icon displays the message: "Congratulations! You are Eligible for the Loan" and "Model Confidence: 92.14%".

The browser console on the right shows the following log entries:

- Download the React DevTools for a better development experience: <https://reactjs.org/link/react-devtools>
- Form submitted (script.js:48)
- Form Data: {gender: 'Male', married: 'Yes', education: 'Graduate', self_employed: 'No', applicant_income: '600000', coapplicant_income: '200000', loan_amount: '500000', loan_amount_term: '180', credit_history: 'Good'}
- Sending POST request to /predict ... (script.js:51)
- Prediction Response: {status: 'success', message: 'Prediction completed successfully', prediction: 'Eligible', confidence: 0.9214} (script.js:56)
- Prediction successful (script.js:56)
- Result page loaded (script.js:57)

Below the log entries, a table titled "PERFORMANCE METRICS" is displayed:

Metric	Value
Request URL	http://127.0.0.1:5000/predict
Status Code	200 OK
Response Time	287 ms
Transfer Size	1.21 KB
DOM Content Loaded	412 ms
Load Complete	535 ms

At the bottom of the console, it states "All systems operational." (script.js:68).